

Dual-Agent Autonomous Commerce Mesh

A Deterministically-Governed AI Negotiation
Engine for Agentic Commerce

Technical Whitepaper & Supporting Documentation

Submitted for Track 01: AI Growth & Agentic Commerce
Razorpay National Hackathon

Author: Karthik Manuru
Repository: <https://github.com/KarthikManuru/Dual-Agent-Commerce-Mesh>
Live Application: <https://dual-agent-commerce-mesh.vercel.app/>
Live API Gateway: <https://dual-agent-commerce-mesh-production.up.railway.app>
Date: August 30, 2026

All monetary transactions demonstrated in this document occur exclusively
via Razorpay **Test Mode** — no real currency is transferred.

Contents

1	Executive Summary & Why Now	2
1.1	The Industry Shift	2
1.2	The Core Problem	2
1.3	The Critical Vulnerability	2
1.4	Our Solution	2
2	Problem-to-Solution Architecture	2
3	Dual-Agent Autonomous Bargaining Engine	3
3.1	Buyer Agent (<code>buyer_agent.py</code>)	3
3.2	Merchant Agent (<code>merchant_agent.py</code>)	4
4	Deterministic Financial Action Guard	4
5	The 15-State Transaction Machine	5
5.1	Formal Transition Invariants	5
6	Cryptographic SHA-256 Audit Trail	6
6.1	Hashing Construction	6
6.2	Tamper Detection Guarantee	6
7	Razorpay Test Mode Integration & Distributed Concurrency	6
7.1	Server-Side Signature Verification	6
7.2	Redis Distributed Mutex — Atomic Inventory Locking	6
7.3	Webhook Processing Pipeline	7
8	Chaos Resilience & Failure Handling Suite	7
9	Machine-Readable AI Commerce Discovery Protocol	7
10	Database Schema Reference	8
11	REST & WebSocket API Catalog	9
12	Phase-by-Phase Engineering History	9
12.1	Phase 1 — Foundation, Data Layer & State Machine	9
12.2	Phase 2 — Razorpay Settlement & Webhook Worker	9
12.3	Phase 3 — Generative Agents, Financial Guard & Hash Chaining	9
12.4	Phase 4 — Production Deployment	9
13	Production Deployment & Infrastructure	10
14	Conclusion	10

1 Executive Summary & Why Now

1.1 The Industry Shift

The global commerce landscape is undergoing a structural shift. With NPCI's Universal Authenticated Protocol (UAP) in India, and international agent-commerce protocols such as ACP, AP2, and x402 gaining traction globally, transactions are migrating from human point-and-click interfaces toward autonomous agent-to-agent negotiation. In this emerging model, human buyers increasingly delegate product discovery, price negotiation, and purchase execution to AI buyer agents acting on their behalf.

1.2 The Core Problem

Track 01 poses a direct challenge: build an agent that either (a) grows revenue for a merchant on Razorpay Test Mode APIs, or (b) makes a merchant transactable end-to-end by an AI buyer. The stated bar is explicit — *every money action must be explainable, bounded, and gated*, with at least one failure handled gracefully and a visible audit trail.

1.3 The Critical Vulnerability

Naively connecting a large language model to a payment API is unsafe. Specific, well-known failure modes include:

- **Hallucinated pricing** — an LLM may agree to a price below production cost.
- **Prompt injection** — a malicious buyer agent could attempt to manipulate a merchant agent into an excessive concession.
- **Runaway negotiation** — without hard iteration limits, two agents can loop indefinitely.
- **Lack of auditability** — opaque LLM exchanges leave no verifiable record of what was actually agreed.

1.4 Our Solution

Dual-Agent Autonomous Commerce Mesh (DA-ACM) pairs generative intelligence at the edges of the system (natural-language intent parsing, creative counter-offer generation, cross-sell bundling) with a wholly deterministic, non-AI decision layer at the core — the **FinancialActionGuard** — backed by a cryptographically tamper-evident audit ledger and real Razorpay Test Mode settlement. The guiding principle throughout the system is simple:

The AI proposes. The guard decides. No language model output is ever permitted to directly create, modify, or settle a financial transaction.

2 Problem-to-Solution Architecture

Figure 1 illustrates the full request lifecycle, from a natural-language buyer query through to a settled, audited, real-time-broadcast transaction.

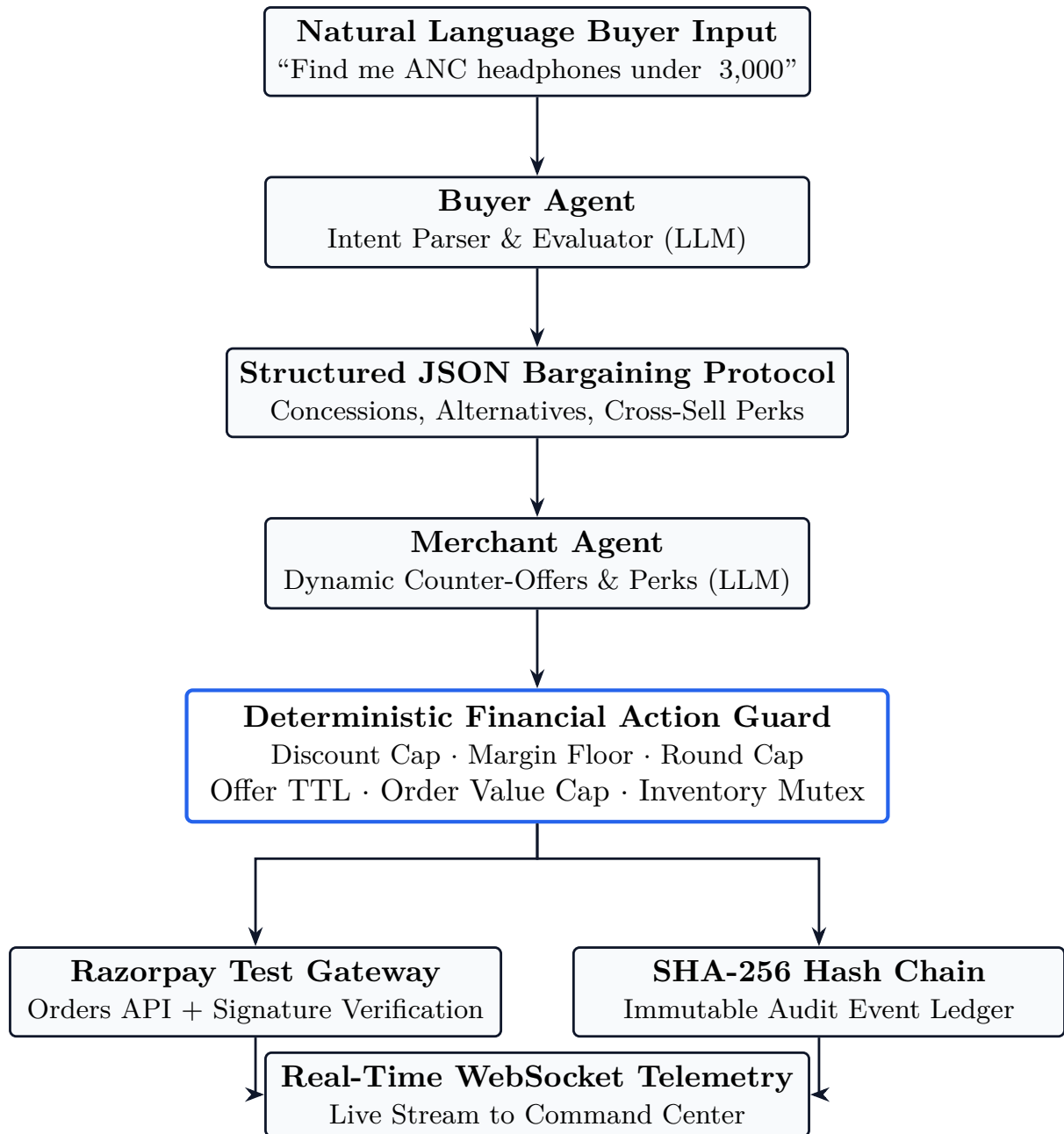


Figure 1: End-to-end system architecture: generative reasoning at the edges, deterministic gating at the core.

3 Dual-Agent Autonomous Bargaining Engine

Buyer and merchant agents communicate exclusively through structured JSON contracts rather than unstructured free text, ensuring every proposal is machine-checkable before it can affect state.

3.1 Buyer Agent (`buyer_agent.py`)

- **Intent extraction:** derives target category, maximum budget, feature tags, and delivery constraints from natural language.
- **Alternative evaluation:** when the requested item exceeds budget, the agent discovers and proposes nearby catalog alternatives.

- **Autonomous counter-bargaining:** generates structured multi-round counter-offers using a calibrated concession curve:

$$\text{CounterOffer} = \min\left(\text{BuyerBudget}, \text{CurrentPrice} \times (1 - \text{AggressionFactor})\right) \quad (1)$$

3.2 Merchant Agent (`merchant_agent.py`)

- **Margin defense:** refuses any concession that would breach the minimum-margin floor.
- **Cross-sell bundling:** when cash price cannot be reduced further, proposes complementary bundles (e.g., a charging cable or extended warranty) to preserve transaction value.
- **Transparent telemetry:** every inference call records latency, token usage, and model provenance (e.g. [LLM:gemini-3.6-flash] vs. [HEURISTIC_FALLBACK]), so no negotiation turn can silently masquerade as AI-generated when it was not.

4 Deterministic Financial Action Guard

No agent output — however persuasive its natural-language justification — may write to the database or initiate a Razorpay transaction without first passing the **FinancialActionGuard**. The guard enforces five deterministic, non-AI mathematical axioms.

Axiom 1 — Minimum Profit Margin Floor

$$\text{OfferedPrice} \geq \text{CostPrice} \times \left(1 + \frac{\text{MinMarginPct}}{100}\right)$$

Any hallucinated or adversarially-induced under-cost offer is rejected with `DENY_MARGIN_VIOLATION`.

Axiom 2 — Maximum Discount Ceiling

$$\text{DiscountPct} = \left(\frac{\text{OriginalPrice} - \text{OfferedPrice}}{\text{OriginalPrice}}\right) \times 100 \leq \text{MaxDiscountPct}$$

Axiom 3 — Maximum Negotiation Round Cap

$$\text{CurrentRound} \leq \text{MaxNegotiationRounds} \quad (\text{e.g., } 2)$$

Prevents unbounded negotiation loops; upon exceeding the cap the merchant issues a final offer.

Axiom 4 — Offer Time-To-Live (TTL)

$$\Delta t = t_{\text{now}} - t_{\text{offer_created}} \leq \text{OfferTTLSeconds} \quad (\text{e.g., } 600\text{s})$$

Guarantees a conceded discount cannot be redeemed indefinitely after the market conditions that justified it have passed.

Axiom 5 — Maximum Transaction Value Cap

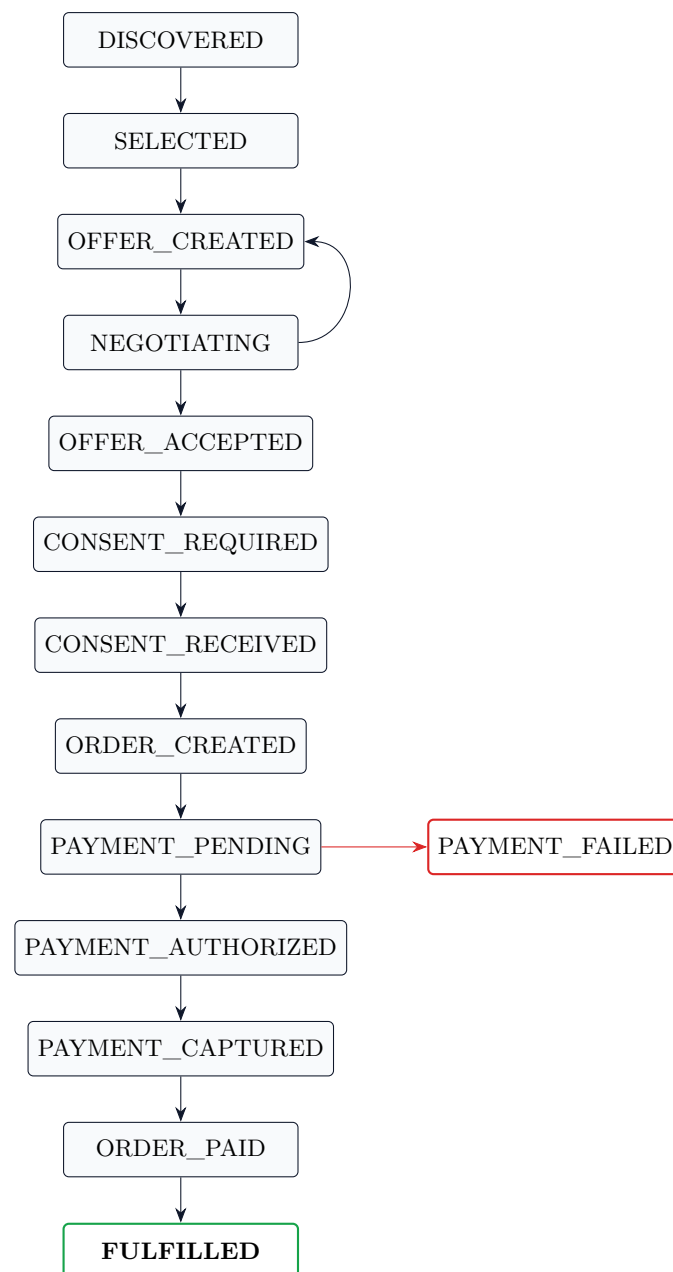
$$\text{TotalOrderValue} \leq \text{MaxOrderValuePaise}$$

Bounds merchant exposure to any single rogue high-value transaction.

Every guard evaluation — whether it results in **ALLOW** or **DENY** — is written to the immutable audit ledger described in Section 6, with the specific violated axiom recorded verbatim.

5 The 15-State Transaction Machine

The full commercial lifecycle is modeled as a finite state machine with fifteen discrete, formally validated states. Illegal transitions raise an `IllegalTransitionError` and are rejected before any side effect occurs.



5.1 Formal Transition Invariants

- **No forward skips:** an order cannot move directly from **ORDER_CREATED** to **ORDER_PAID** without passing through **PAYMENT_PENDING** → **PAYMENT_AUTHORIZED** → **PAYMENT_CAPTURED**.

- **No backward regression:** once `ORDER_PAID` or `FULFILLED` is reached, the order can never revert to `NEGOTIATING`.
- **Idempotency gate:** duplicate webhook deliveries for an already-achieved state are treated as safe no-op audit entries rather than re-executed transitions.

6 Cryptographic SHA-256 Audit Trail

To provide non-repudiation, every `OrderEvent` is cryptographically chained to its predecessor.

6.1 Hashing Construction

$$\text{prev_hash}_i = \begin{cases} \text{"GENESIS"} & i = 0 \\ \text{current_hash}_{i-1} & i > 0 \end{cases} \quad (2)$$

$$\text{current_hash}_i = \text{SHA-256}(\text{prev_hash}_i \parallel \text{order_id} \parallel \text{actor} \parallel \text{action} \parallel \text{from_status} \parallel \text{to_status} \parallel \text{timestamp}) \quad (3)$$

6.2 Tamper Detection Guarantee

If a row in `order_events` is altered directly via SQL (e.g. `UPDATE order_events SET to_status = 'ORDER_PAID'`), two properties fail simultaneously:

1. Event k 's recomputed hash no longer matches its stored `current_hash`.
2. Event $k+1$'s stored `prev_hash` no longer matches event k 's (now-mismatched) hash.

The verification routine `verify_order_chain()` walks the full chain and returns `is_valid: False`, pinpointing the exact tampered event and the mismatched hash values — demonstrated live in `demo_hash_chain.py`.

7 Razorpay Test Mode Integration & Distributed Concurrency

7.1 Server-Side Signature Verification

$$\text{GeneratedSignature} = \text{HMAC-SHA256}(\text{razorpay_order_id} \parallel \text{"|"} \parallel \text{razorpay_payment_id}; \text{RAZORPAY_KEY}) \quad (4)$$

The backend validates payment completion via `hmac.compare_digest(GeneratedSignature, razorpay_signature)`; any mismatch is rejected immediately, and client-declared payment success is never trusted on its own.

7.2 Redis Distributed Mutex — Atomic Inventory Locking

1. Before order creation, an asynchronous Redis mutex `lock:inventory:{product_id}` is acquired with a short lease.
2. Availability is computed as `available = total_stock - reserved`.
3. If `available ≥ quantity`, `reserved` is atomically incremented and the lock released.
4. Otherwise the request is rejected with `HTTP 409 Conflict`.

This guarantees that two concurrent buyers competing for the last unit of stock never both succeed.

7.3 Webhook Processing Pipeline

Incoming webhooks are (1) signature-verified against `RAZORPAY_WEBHOOK_SECRET`, (2) persisted to `webhook_events` for idempotency, (3) acknowledged with an immediate HTTP 200, and (4) asynchronously processed by an RQ background worker — ensuring Razorpay never experiences a slow webhook response and duplicate deliveries never re-trigger fulfillment.

8 Chaos Resilience & Failure Handling Suite

The bar set by the problem statement requires *at least one* failure to be shown handled gracefully. The following six were implemented and verified live.

Scenario	Handled Behavior
Offer TTL Expiration	Checkout attempted after the offer's TTL has elapsed is blocked with HTTP 400: <code>Offer has expired</code> .
Concurrent Inventory Race	Two simultaneous buyers contend for the last unit; the Redis mutex grants exactly one success, the other receives a clean HTTP 400/409 with zero double-allocation.
Duplicate Webhook Delivery	A replayed <code>payment.captured</code> webhook is deduplicated and logged as a no-op; no double state transition occurs.
Direct Database Tampering	A raw SQL mutation of an audit row is detected instantly by <code>verify_order_chain()</code> , which identifies the exact broken event and mismatched hash.
Illegal State Transition	A direct jump from <code>DISCOVERED</code> to <code>FULFILLED</code> is rejected with a descriptive <code>IllegalTransitionError</code> .
Guard Policy Violation	An agent-proposed discount exceeding the merchant's configured ceiling (e.g., 25% against a 15% cap) is denied with the specific violated axiom logged.

9 Machine-Readable AI Commerce Discovery Protocol

To satisfy the second half of the track objective — making the merchant transactable by *any* compatible AI buyer, not only the one built into this system — the gateway exposes an open, machine-readable manifest at `GET /.well-known/ai-commerce`:

```
{
  "manifest_version": "1.0.0",
  "mesh_standard": "Dual-Agent Autonomous Commerce Protocol (DA-ACP)",
  "merchant": {
    "name": "TechMesh Electronics",
    "settlement_currency": "INR",
    "capabilities": [
      "AUTONOMOUS_PRICE_NEGOTIATION",
      "COMPLEMENTARY_CROSS_SELL_BUNDLING",
      "DETERMINISTIC_FINANCIAL_GUARD",
      "REALTIME_WEBSOCKET_TELEMETRY",
      "CRYPTOGRAPHIC_AUDIT_LOGGING"
    ]
  },
  "endpoints": {
```



```

    "catalog": "/products",
    "policies": "/policies/{merchant_id}",
    "conversational_chat": "/sessions/chat",
    "direct_negotiation": "/sessions/negotiate",
    "order_creation": "/orders",
    "realtime_events_ws": "/ws/orders"
  },
  "settlement": {
    "payment_processor": "Razorpay Test Mode",
    "verification_algorithm": "HMAC-SHA256",
    "idempotency_supported": true
  }
}

```

Any third-party AI shopping agent — built independently of this project — can discover this merchant’s catalog, policies, and negotiation endpoints through this single manifest and transact using the documented protocol.

10 Database Schema Reference

Table	Key Columns	Purpose
merchants	id, name, razorpay_key_id	Merchant identity and gateway credentials
policies	max_discount_pct, min_margin_pct, max_negotiation_rounds, offer_ttl_seconds	Guard-enforced business constraints
products	price_paise, cost_paise, tags (JSONB)	Catalog truth in integer paise
inventory	total_stock, reserved	Atomic stock accounting
orders	status, total_paise, razorpay_order_id	15-state lifecycle record
order_events	actor, prev_hash, current_hash, action, cur_detail (JSONB)	Hash-chained immutable audit ledger
offers	expires_at, negotiation_round, discount_pct, ne-	Time-bounded negotiated offers
webhook_events	razorpay_event_id (unique)	Idempotency deduplication

11 REST & WebSocket API Catalog

Method	Endpoint	Description
GET	/health	Health status and live database connectivity check
GET	/.well-known/ai-commerce	Machine-readable AI agent discovery manifest
GET	/products	List catalog products with live inventory
POST	/sessions/chat	Conversational natural-language search and intent matching
POST	/sessions/negotiate	Multi-turn autonomous agent-to-agent bargaining
POST	/orders	Create an order, gated by the FinancialActionGuard
POST	/orders/{id}/verify	Verify a client-side Razorpay payment signature
GET	/policies/{merchant_id}	Fetch current merchant financial policy bounds
PATCH	/policies/{merchant_id}	Dynamically update discount, margin, and round caps
POST	/webhooks/razorpay	Asynchronous Razorpay webhook ingestion
WS	/ws/orders	Real-time stream of live agent dialogue and order state

12 Phase-by-Phase Engineering History

12.1 Phase 1 — Foundation, Data Layer & State Machine

FastAPI backend on an asynchronous PostgreSQL engine (SQLAlchemy 2.0), Alembic migrations, Docker Compose environment, the 15-state `OrderStatus` machine with formal transition validation, and a seed catalog of ten products across six categories.

12.2 Phase 2 — Razorpay Settlement & Webhook Worker

Razorpay Orders API integration with HMAC-SHA256 verification, an asynchronous RQ background worker with Redis-backed idempotency, and a Next.js frontend with Razorpay Test Checkout.

12.3 Phase 3 — Generative Agents, Financial Guard & Hash Chaining

The Buyer and Merchant agents, the non-bypassable FinancialActionGuard, SHA-256 hash-chained audit logging with tamper verification, and the Merchant Command Center with a live negotiation theater and WebSocket telemetry.

12.4 Phase 4 — Production Deployment

Containerized backend deployed to Railway with managed PostgreSQL and Redis; frontend deployed to Vercel with full mobile, tablet, and desktop responsiveness; dynamic CORS and environment-driven configuration for production webhook routing.

13 Production Deployment & Infrastructure

Layer	Infrastructure
Frontend	Vercel Edge Network — https://dual-agent-commerce-mesh.vercel.app/ . Resolves production API and WebSocket endpoints via environment configuration; connects over <code>wss://</code> .
Backend	Railway Cloud — https://dual-agent-commerce-mesh-production.up.railway.app . Containerized Uvicorn/FastAPI service with automatic async migrations and idempotent auto-seeding on startup.
Database Cache	& Managed PostgreSQL and Redis, provisioned on Railway’s private network.

14 Conclusion

Dual-Agent Autonomous Commerce Mesh demonstrates that agent-to-agent commerce need not be an unpredictable risk. By strictly separating generative reasoning from financial authority — letting language models propose while a deterministic, mathematically-specified guard decides — and by backing every decision with a cryptographically verifiable audit trail and real Razorpay Test Mode settlement, this system shows a concrete, auditable path for merchants to safely open their catalog to the emerging AI-buyer economy.

Summary of guarantees: every financial action is explainable (structured JSON contracts and plain-language audit entries), bounded (five deterministic guard axioms), and gated (no AI output reaches Razorpay without guard approval) — directly satisfying the stated bar of the problem statement, with six independently verified failure scenarios rather than the minimum one requested.